

Next.js Pages Router -> App Router 전환 분석 보고서

작성일: 2026-06-04

대상 저장소: NextJs_Main

기준 브랜치: main

문서 전제

- 본 보고서는 사용자가 요청한 "기존 Pages Router 기반 프로젝트의 App Router 전환 분석" 형식을 따르되, 실제 저장소를 확인한 결과를 기준으로 작성했다.
- 현재 저장소에는 `pages/` 디렉터리와 Pages Router 전용 API 사용 흔적이 확인되지 않았다.
- 따라서 이 문서는 "Pages Router -> App Router 전환 계획"이라기보다, "이미 App Router 기반으로 운영 중인 프로젝트의 전환 완료 상태 점검 및 안정화 계획"에 가깝다.
- 분석 md 원문이 별도로 제공되지 않았으므로, 추측이 필요한 항목은 `가정` 또는 `확인 필요`로 명시했다.

1. 요약 결론

- 현재 프로젝트 상태:
 - `Next.js 15.5.14` 기반의 App Router 프로젝트다.
 - 실제 라우트는 `app/` 디렉터리 기준으로 구성되어 있다.
 - `pages/`, `getStaticProps`, `getServerSideProps`, `getStaticPaths`, `next/head`, `next/router` 사용 흔적은 현재 저장소에서 확인되지 않았다.
- App Router 전환 가능성:
 - 전환 가능성을 평가하는 단계는 이미 지났고, 현재는 App Router 운영 품질 보장 단계로 보는 것이 맞다.
- 전체 전환 / 점진 전환 추천:
 - 전체 전환은 불필요하다.
 - 점진적 안정화가 적합하다.
- 가장 큰 리스크:
 - 현재 상태를 Pages Router 프로젝트로 오인하고 Main 브랜치에서 구조를 크게 변경하는 것.
 - 실제로는 App Router가 이미 운영 중이므로, 잘못된 전환 작업은 URL, metadata, layout, API 동작을 깨뜨릴 수 있다.
- 먼저 해야 할 작업 5개:
 - 현재 운영 URL과 실제 라우트 목록을 확정한다.
 - Main 직접 수정 대신 feature branch + PR + Vercel Preview 흐름을 강제한다.
 - `app/not-found.tsx`, `app/error.tsx`, `app/loading.tsx` 최소 구조를 추가한다.
 - global metadata, Open Graph, canonical, `robots.ts`, `sitemap.ts`를 정비한다.
 - `/api/send-email`와 관련된 Vercel 환경 변수를 검증한다.

2. 현재 구조 분석

현재 저장소는 App Router 기반 포트폴리오/콘텐츠 사이트다.

현재 확인된 주요 라우트

- `/` -> `app/page.tsx`
- `/about` -> `app/about/page.tsx`
- `/projects` -> `app/projects/page.tsx`

- `/contact` -> `app/contact/page.tsx`
- `/login` -> `app/login/page.tsx`
- `/signup` -> `app/signup/page.tsx`
- `/profile` -> `app/profile/page.tsx`
- `/music-list` -> `app/music-list/page.tsx`
- `/DJ_Play_List` -> `app/DJ_Play_List/page.tsx`
- `/tax-calculator` -> `app/tax-calculator/page.tsx`
- `/blog` -> `app/blog/page.tsx`
- `/blog/[slug]` -> `app/blog/[slug]/page.tsx`
- `/admin` -> `app/admin/page.tsx`
- `/admin/users` -> `app/admin/users/page.tsx`
- `/api/send-email` -> `app/api/send-email/route.ts`

공통 구조

- 루트 레이아웃은 `app/layout.tsx` 에서 처리한다.
- 루트 layout에서 `NavBar` , `Footer` , `BottomNav` , `AuthProvider` , `globals.css` 를 연결한다.
- admin 하위는 별도 `app/admin/layout.tsx` 를 사용한다.
- 블로그 상세는 `generateStaticParams` , `generateMetadata` , `notFound()` 를 사용한다.

데이터 구조

- 인증: `context/AuthContext.tsx` + `localStorage`
- 블로그 데이터: `data/blogPosts.ts`
- 음악 데이터: `data/musicData.ts`
- 세금 계산 데이터: `app/config/taxRates2025.ts`
- 순수 유틸/계산 로직: `app/lib/taxCalculator.ts` , `lib/utills.ts` , `utills/Utills.ts`

확인 결과 요약

- `pages/` 디렉터리: 없음
- `middleware.ts` : 없음
- `hooks/` : 없음
- `services/` : 없음
- `pages/api/*` : 없음
- 현재 워크스페이스 오류: 없음

3. Pages Router 의존 항목

아래 표는 요청된 항목 기준으로 실제 저장소 상태를 정리한 것이다.

항목	현재 상태	판정	비고
<code>pages/index.tsx</code>	없음	확인 결과 없음	현재는 <code>app/page.tsx</code> 사용
<code>pages/[slug].tsx</code>	없음	확인 결과 없음	현재는 <code>app/blog/[slug]/page.tsx</code> 사용
<code>pages/api/*</code>	없음	확인 결과 없음	현재는 <code>app/api/send-email/route.ts</code> 사용

pages/_app.tsx	없음	확인 결과 없음	현재는 app/layout.tsx 사용
pages/_document.tsx	없음	확인 결과 없음	App Router 기준 별도 관리 없음
pages/404.tsx	없음	확인 결과 없음	현재 app/not-found.tsx도 없음
pages/500.tsx	없음	확인 결과 없음	현재 app/error.tsx도 없음
getStaticProps	없음	미사용	App Router 직접 데이터 로딩으로 대체
getStaticPaths	없음	미사용	generateStaticParams 사용
getServerSideProps	없음	미사용	현재 저장소에서 SSR 전용 API 미확인
next/head	없음	미사용	metadata, generateMetadata 사용
next/router	없음	미사용	next/navigation 사용
next/link	있음	정상 사용	App Router에서도 그대로 사용 가능
next/image	있음	정상 사용	App Router에서도 그대로 사용 가능
public/	있음	유지 가능	정적 자산 유지
styles/	있음	유지 가능	globals.css 사용
components/	있음	유지 가능	공통 UI/레이아웃 컴포넌트
lib/	있음	유지 가능	유틸 함수 위치
utils/	있음	유지 가능	보조 함수 위치
hooks/	없음	확인 필요 아님	현재 저장소 기준 없음
services/	없음	확인 필요 아님	현재 저장소 기준 없음
middleware.ts	없음	확인 필요 아님	현재 저장소 기준 없음
next.config.js 또는 next.config.mjs	없음	해당 없음	next.config.ts 사용
next.config.ts	있음	사용 중	reactStrictMode, image domain 설정

결론

- 현재 저장소는 Pages Router 의존 지점이 사실상 없다.
- 따라서 전환 포인트는 Pages Router 제거가 아니라 App Router 운영 품질 강화다.

4. App Router 전환 매핑표

주의:

- 아래 표의 기존 구조 는 일반적인 Pages Router 기준 매핑이다.
- 현재 저장소에 실존하는 파일이 아닌 경우, "가정" 또는 "확인 결과 없음" 전제로 읽어야 한다.

기존 구조	App Router 구조	전환 난이도	리스크	조치 방안
pages/index.tsx	app/page.tsx	낮음	낮음	현재 구조 유지
pages/about.tsx	app/about/page.tsx	낮음	낮음	현재 구조 유지
pages/projects.tsx	app/projects/page.tsx	낮음	낮음	현재 구조 유지
pages/blog/index.tsx	app/blog/page.tsx	낮음	낮음	현재 구조 유지
pages/blog/[slug].tsx	app/blog/[slug]/page.tsx	중간	metadata, 404 처리 누락 가능	generateStaticParams, generateMetadata, notFound() 유지
pages/_app.tsx	app/layout.tsx	중간	provider 범위가 과도해질 수 있음	server layout 유지, client provider 범위 최소화 검토
pages/_document.tsx	app/layout.tsx의 html, body	낮음	낮음	현재 구조 유지
pages/404.tsx	app/not-found.tsx	낮음	404 UX 및 SEO 저하	파일 추가 권장
pages/500.tsx	app/error.tsx	낮음	런타임 예외 대응 부족	파일 추가 권장
pages/api/*	app/api/*/route.ts	낮음	env, runtime, validation 누락	현재 send-email은 이미 완료
next/head	metadata, generateMetadata	중간	canonical, OG 누락	전역 metadata 보강
next/router	next/navigation	낮음	client 경계 과대화	client-only 영역만 유지
getStaticProps	Server Component 데이터 로딩 / revalidate	중간	내부 API self-fetch 오남용	정적 데이터는 직접 import 유지

getStaticPaths	generateStaticParams	낮음	slug 누락	블로그에 이미 적용됨
getServerSideProps	동적 fetch / no-store / cookies / headers	중간	캐시 정책 옵션	현재 저장소에서는 미사용

5. 권장 디렉터리 구조

현재 저장소는 규모상 단순 App Router 구조 와 가벼운 feature-based 구조 의 혼합이 가장 적합하다.

추천 구조 유형

- 1. 단순 App Router 구조: 가능
- 2. feature-based 구조: 가장 적합
- 3. domain-based 구조: 현재 규모에서는 과도할 수 있음
- 4. content-driven 구조: 블로그 비중이 커지면 고려 가능
- 5. Next.js + 외부 API 서버 분리 구조: 현재 단계에서는 불필요

권장 디렉터리 구조 예시

```

app/
  layout.tsx
  page.tsx
  not-found.tsx
  loading.tsx
  error.tsx
  about/
    page.tsx
  blog/
    page.tsx
    [slug]/
      page.tsx
      loading.tsx
  contact/
    page.tsx
  projects/
    page.tsx
  music-list/
    page.tsx
  dj-play-list/
    page.tsx
  tax-calculator/
    page.tsx
  login/
    page.tsx
  signup/
    page.tsx
  profile/
    page.tsx

```

```
admin/
  layout.tsx
  page.tsx
  users/
    page.tsx
api/
  send-email/
    route.ts
robots.ts
sitemap.ts

components/
  layout/
  navigation/
  shared/
  ui/

features/
  auth/
    components/
    context/
    utils/
  blog/
    components/
    data/
  music/
    components/
    data/
  contact/
    components/
    api/
  tax/
    components/
    utils/

lib/
services/
types/
styles/
public/
```

구조 관련 권장안

- 현재 `DJ_Play_List` 는 URL/디렉터리 명명 규칙이 일관적이지 않다.
- 장기적으로는 `/dj-play-list` 로 정규화하는 것이 좋다.
- 단, 운영 URL이 이미 공개되어 있다면 즉시 변경하지 말고 `redirect`와 `canonical`부터 설계해야 한다.

6. Server Component / Client Component 분리안

App Router에서는 기본적으로 Server Component 중심 설계를 유지하는 것이 좋다. 현재 저장소를 기준으로 분류하면 다음과 같다.

파일/기능	Server 가능 여부	Client 필요 여부	이유	권장 조치
루트 layout	가능	불필요	레이아웃 자체는 서버 렌더링 가능	현재처럼 Server 유지
NavBar	불가	필요	useAuth, usePathname, useRouter 사용	Client 유지
BottomNav	불가	필요	usePathname 사용	Client 유지
Footer	가능	불필요에 가까움	혹, 브라우저 API, 상태 없음	Server Component 전환 검토
Hero	불가	필요	GSAP, useEffect 사용	Client 유지
BlogPage	가능	불필요	정적 데이터 렌더링	Server 유지
BlogPostPage	가능	불필요	정적 데이터 + metadata + 404 처리	Server 유지
ContactPage	부분 가능	필요	form state, submit 상태, preview	Client 유지
LoginPage	불가	필요	localStorage auth, form state, router push	Client 유지
SignupPage	불가	필요	form state, auth interaction	Client 유지
ProfilePage	불가	필요	useAuth, useEffect, router push	Client 유지
MusicListPage	불가	필요	Audio API, useRef, 상태 관리	Client 유지
DJ_Play_List	불가	필요	Audio API, 이벤트 핸들링	Client 유지
TaxPage	불가	필요	form interaction, state, 계산 결과 표시	Client 유지
AdminLayout	부분 가능	현재는 필요	모바일 drawer 상태 사용	Server shell + client menu 분리 검토
About, Projects	가능	불필요 추정	정적 콘텐츠 성격	Server 유지 권장

Client Component로 강제되는 주요 영역

- 인증 UI 전반
- 오디오 플레이어 전반
- 세금 계산기
- 연락 폼
- pathname 기반 네비게이션

추가 권장안

- Footer 는 Client Component일 필요성이 낮다.

- AdminLayout 은 전체를 Client로 둘 필요가 약하므로, 상호작용 부분만 Client로 격리하면 hydration 범위를 줄일 수 있다.

7. 데이터 패칭 전환안

현재 프로젝트는 Pages Router의 `getStaticProps` / `getServerSideProps` 기반이 아니라, 정적 TypeScript 데이터 import + 일부 클라이언트 fetch 구조다.

데이터 유형	기존 방식	App Router 권장 방식	cache/revalidate 전략	주의사항
블로그 목록	<code>data/blogPosts.ts</code> 직접 import	Server Component에서 직접 import	빌드 타임 정적	CMS 도입 전까지 가장 안정적
블로그 상세	<code>getPostBySlug()</code> + <code>generateStaticParams</code>	현재 방식 유지	정적 생성, 필요 시 revalidate 검토	slug 누락 시 <code>notFound()</code> 유지
음악 목록	<code>data/musicData.ts</code> 를 Client page에서 직접 사용	데이터는 Server에서 읽고 player만 Client로 분리 가능	정적	데이터가 커지면 bundle 증가 주의
세금표	정적 config import	현재 방식 유지	정적	계산식은 pure function 유지
연락 메일 전송	Client <code>fetch('/api/send-email')</code>	현재 유지 또는 Server Action 검토	no-store 성격	서버 검증과 abuse 방어 필요
인증 상태	localStorage	Client-only 유지	캐시 없음	서버 인증으로 전환 전까지 Client 고정

요청 항목별 판단

- `getStaticProps` : 확인 결과 없음
- `getServerSideProps` : 확인 결과 없음
- `getStaticPaths` : 확인 결과 없음, 대신 `generateStaticParams` 사용
- CSR fetch: `app/contact/page.tsx` 에서 확인
- SWR / React Query 사용 여부: 확인 결과 없음
- 내부 API 호출 방식: `/api/send-email` 1건 확인
- 외부 API 호출 방식: 직접적인 데이터 API 호출은 확인 결과 없음
- DB 직접 접근 여부: 확인 결과 없음
- Markdown/MDX 파일 로딩 여부: 확인 결과 없음, 블로그는 TS 문자열 데이터 기반
- 빌드 타임 생성 여부: 블로그 상세 SSG 확인
- 런타임 서버 렌더링 여부: 현재 저장소에서 명시적 동적 fetch 미확인
- ISR 필요 여부: 현재는 명시적으로 미사용
- 캐시 정책 필요 여부: 메일 전송 API에만 no-store 성격이 강함

권장안

- 블로그는 현재 구조가 적절하다.
- 음악 목록은 서버 렌더링 가능한 데이터 출력과 클라이언트 플레이어를 분리하면 더 좋다.
- 연락 폼은 Route Handler 유지가 충분히 합리적이며, Server Action 전환은 선택 사항이다.

8. API Routes 전환안

현재 저장소에서 확인된 API는 1개다.

기존 API	역할	App Router 전환 여부	대안	리스크
/api/send-email	연락 폼 메일 발송	이미 app/api/send-email/route.ts로 전환 완료	Server Action 또는 외부 메일 서비스	env 누락, 스팸, rate limit 부재, SMTP 장애

세부 판단

- 유지해도 되는 API:
 - /api/send-email
- app/api/*/route.ts 로 옮겨야 하는 API:
 - 현재 추가 대상 없음
- 외부 백엔드로 분리하는 게 나은 API:
 - 현재 규모에서는 없음
 - 메일 발송량이 늘면 Resend, SendGrid, SES 등 외부 서비스 검토 가능
- Vercel Serverless Function 비용/제한 주의 API:
 - 메일 발송 API는 SMTP 응답 지연과 timeout 가능성에 주의해야 함
- 인증/세션/쿠키 처리 API:
 - 확인 결과 없음
- 파일 업로드/이미지 처리 API:
 - 확인 결과 없음
- 로그 수집 API:
 - 확인 결과 없음
- 관리자 API:
 - 확인 결과 없음

권장안

- 우선순위는 패턴 변경보다 운영 안정성이다.
- 메일 전송 API에는 최소한 입력 검증, 에러 처리, rate limit, 스팸 방지 전략이 필요하다.

9. SEO / Metadata 전환안

현재 SEO는 부분 적용 상태다. 루트 metadata와 블로그 metadata는 있으나, 운영 수준의 SEO 세트는 아직 부족하다.

SEO 항목	기존 방식	App Router 권장 방식	우선 순위	조치
title	일부 있음	metadata 유지	P1	주요 페이지 기본값 정리

description	일부 있음	metadata 유지	P1	페이지별 설명 보강
Open Graph	확인 결과 없음	openGraph 추가	P1	기본 OG 이미지 포함
Twitter Card	확인 결과 없음	twitter 추가	P1	OG와 함께 설정
canonical URL	확인 결과 없음	metadataBase + canonical 설정	P1	블로그 상세에 특히 중요
robots	확인 결과 없음	app/robots.ts 추가	P1	preview/prod 정책 분리 검토
sitemap	확인 결과 없음	app/sitemap.ts 추가	P1	블로그 slug 포함
favicon	있음	유지	P2	필요 시 아이콘 세트 확장
viewport	명시적 설정 미확인	필요 시 viewport export	P2	기본값으로 충분할 가능성 높음
페이지별 동적 metadata	블로그 상세만 있음	generateMetadata 유지	P1	canonical, OG 확장
블로그/콘텐츠 상세 metadata	부분 적용	title/description/OG 확장	P1	검색/공유 품질 향상
JSON-LD 구조화 데이터	확인 결과 없음	Article schema 검토	P2	블로그 비중이 커질 때 유효

현재 누락된 App Router 운영 파일

- `app/not-found.tsx` : 없음
- `app/error.tsx` : 없음
- `app/loading.tsx` : 없음

권장안

- metadata만 보강하는 것이 아니라, 에러/로딩/404 구조까지 함께 정비해야 App Router 전환 품질이 완성된다.

10. Vercel 배포 영향

가정:

- Production Branch는 `main` 일 가능성이 높다.
- 실제 Vercel 프로젝트 설정은 본 저장소만으로는 확정할 수 없다.

항목별 검토

- Production Branch:
 - `main` 직접 배포 구조일 가능성이 높다.
 - 구조 변경 작업은 직접 push보다 PR merge를 강제하는 것이 안전하다.

- Preview Branch:
 - 모든 구조 변경은 preview 배포를 전제로 진행해야 한다.
- Preview URL:
 - preview에서 canonical, robots, metaDataBase가 production URL을 잘못 가리키지 않도록 주의해야 한다.
- Custom Domain:
 - 현재 도메인 설정은 확인 필요.
 - URL 변경 시 redirect와 canonical이 함께 정리되어야 한다.
- Environment Variables:
 - 확인이 필요한 주요 값:
 - SMTP_HOST
 - SMTP_PORT
 - SMTP_SECURE
 - SMTP_USER
 - SMTP_PASS
 - RECEIVER_EMAIL
- Build Command:
 - 기본 next build 로 충분하다.
- Output 설정:
 - 별도 custom output 필요성은 현재 낮다.
- Serverless Function 사용량:
 - /api/send-email 1개라 사용량 부담은 크지 않다.
- Image Optimization 비용 가능성:
 - 외부 shield 이미지를 next/image 로 사용하고 있으므로 요청량 증가 시 비용/제약 가능성은 있다.
- ISR / Cache 영향:
 - 현재는 정적 데이터 위주라 영향이 크지 않다.
- Edge Runtime 사용 여부:
 - 확인 결과 없음
- Middleware 영향:
 - 없음
- 배포 실패 가능 지점:
 - 환경 변수 누락
 - URL 변경 후 redirect 미설정
 - metaDataBase 누락
 - route handler runtime 이슈
 - lint/type 에러
- 롤백 방식:
 - Vercel 이전 배포 복원 + PR revert가 가장 안전하다.

11. Main 브랜치 단일 운영 기준 브랜치 전략

현재 Main 브랜치만 운영 중이라는 가정에서, 직접 수정은 지양해야 한다.

전략 항목	권장안	이유	리스크	실행 방법
-------	-----	----	-----	-------

Main 직접 수정 위험도	높음	운영 브랜치일 가능성이 높음	실수 즉시 배포 영향	Main 직접 수정 금지 수준으로 운영
migration/app-router 브랜치 생성	장기 구조 작업에는 권장	안정화 작업 묶음 관리에 유리	브랜치 장수화	migration/app-router-stabilization 권장
기능 단위 브랜치 분리	권장	작은 단위 검증 가능	병합 충돌	feat/seo-metadata, feat/error-boundaries 등으로 분리
Vercel Preview Deployment 활용	필수	URL, metadata, UI, API 검증에 직접적	미사용 시 prod 위험	모든 PR에서 preview 확인
기존 운영 URL 보호 전략	필수	공개 URL이 깨지면 SEO/사용자 영향 큼	404, 링크 손실	URL 목록 동결 문서화
App Router 전환 중 Pages Router 병행 가능 여부	기술적으로는 가능	Next.js는 병행 가능	경로 충돌	현재 저장소에는 불필요
전체 전환 vs 점진 전환 판단	점진 전환	이미 App Router 상태	과도한 리팩터링 위험	작은 PR로 진행
롤백 전략	PR revert + Vercel rollback	가장 빠르고 단순	준비 없으면 복구 지연	사전 롤백 절차 문서화
PR 리뷰 체크리스트	필요	누락 방지	검증 누락 시 운영 영향	템플릿화 권장
배포 전 테스트 항목	필요	실제 영향 지점이 명확	미검증 배포	route smoke + metadata + API 테스트

실무 권장안

- Main 단일 운영은 유지하더라도, 작업 방식은 반드시 브랜치 기반으로 전환해야 한다.
- 이 프로젝트는 데이터베이스 마이그레이션이 없으므로 작은 PR 전략이 특히 잘 맞는다.

12. P0 / P1 / P2 우선순위

우선 순위	항목	이유	예상 작업	검증 방법
P0	Main 직접 수정 중단	운영 리스크 큼	브랜치/PR 정책 수립	Preview-only 검증
P0	운영 URL 동결 및 redirect 정책 수립	URL 깨지면 즉시 영향	라우트 목록 문서화	주요 경로 수동 점검

P0	Vercel 환경 변수 검증	메일 API 장애 가능	env 점검	Preview/Prod 메일 테스트
P0	실제 legacy Pages 브랜치 존재 여부 확인	전제 오픈 방지	브랜치/이전 배포 조사	브랜치 비교
P1	not-found, error, loading 추가	App Router 운영 품질 향상	파일 추가	404/에러/로딩 검증
P1	metadata 확장	SEO 영향 큼	OG, Twitter, canonical, metadataBase	page source 검토
P1	robots.ts, sitemap.ts 추가	검색엔진 운영 기본	파일 추가	실제 URL 확인
P1	Client boundary 축소	성능/유지보수 개선	Footer server화, admin layout 분리	hydration 확인
P1	/DJ_Play_List URL 정책 확정	SEO/일관성 문제	유지 또는 redirect 포함 변경	redirect 테스트
P2	feature 구조 정리	유지보수성 향상	features/* 재배포	코드 리뷰
P2	블로그 콘텐츠 관리 방식 개선	장기 확장성	MDX/CMS 검토	콘텐츠 추가 테스트
P2	localStorage auth 재검토	보안/확장성 한계	Auth.js 또는 외부 백엔드 검토	인증 플로우 테스트

13. 단계별 마이그레이션 계획

현재 저장소 기준으로 "전환"보다 "App Router 안정화" 계획으로 실행하는 것이 맞다.

Phase 0. 현황 정리

- 현재 라우트 목록 정리
- API 목록 정리
- 공통 Layout 구조 확인
- SEO 처리 방식 확인
- 데이터 패칭 방식 확인
- legacy Pages Router 코드가 다른 브랜치에 존재하는지 확인

Phase 1. App Router 기반 최소 구조 생성

현재 이미 존재:

- `app/layout.tsx`
- `app/page.tsx`

추가 권장:

- `app/not-found.tsx`
- `app/loading.tsx`
- `app/error.tsx`

- global style 연결 점검
- metadata 기본값 확장
- `metadataBase` 설정

Phase 2. 정적 페이지 이전

현재 저장소 기준 대부분 이미 이전 완료 상태다.

대상:

- 홈
- 소개
- 포트폴리오
- 블로그 목록
- 기타 정적 페이지

실행 포인트:

- 가능한 Server Component 유지
- 페이지별 metadata 보강
- 불필요한 Client Component 축소

Phase 3. 동적 라우트 이전

현재 `/blog/[slug]` 는 이미 App Router 방식이다.

실행 포인트:

- `[slug]` 상세 페이지 metadata 확장
- `generateStaticParams` 유지
- `loading.tsx` 필요 여부 판단
- not-found 처리 UX 개선

Phase 4. API 이전

현재 `pages/api` 는 없고 Route Handler가 이미 사용 중이다.

실행 포인트:

- `/api/send-email` 검증 강화
- env 문서화
- 인증/쿠키/세션 처리 요구사항은 확인 필요

Phase 5. SEO / 성능 / 배포 검증

검증 대상:

- metadata
- sitemap
- robots
- Open Graph
- Lighthouse
- Vercel Preview
- Production 배포 전 체크

Phase 6. Pages Router 제거 여부 판단

현재 저장소 기준:

- Pages Router 잔존 가능 여부: 없음
- 제거 시점: 해당 없음
- 롤백 가능성: PR revert + Vercel rollback으로 충분
- 최종 구조 정리: App Router 안정화 후 디렉터리 재정리

14. 개발자 실행 체크리스트

App Router 전환 체크리스트

P0

- [] 현재 저장소 외에 legacy Pages Router 코드가 있는 브랜치/배포가 있는지 확인한다.
- [] Main 직접 수정 금지 규칙을 정하고 feature branch + PR + Vercel Preview 흐름으로 전환한다.
- [] 현재 운영 URL 목록을 확정하고 변경 금지 또는 redirect 정책을 문서화한다.
- [] Vercel 환경 변수(SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, RECEIVER_EMAIL)를 점검한다.
- [] `/api/send-email`를 Preview와 Production에서 실제로 전송 테스트한다.

P1

- [] `app/not-found.tsx`를 추가한다.
- [] `app/error.tsx`를 추가한다.
- [] `app/loading.tsx`를 추가한다.
- [] root metadata에 `metadataBase`, Open Graph, Twitter, canonical 기본값을 추가한다.
- [] `app/robots.ts`와 `app/sitemap.ts`를 추가한다.
- [] 블로그 상세 `generateMetadata`에 canonical/OG를 확장한다.
- [] `Footer`를 Server Component로 내릴 수 있는지 검토한다.
- [] `AdminLayout`을 server shell + client menu로 분리할지 검토한다.
- [] `/DJ\_Play\_List` URL을 유지할지 `/dj-play-list`로 정규화할지 결정한다.

P2

- [] `features/` 중심으로 auth, blog, music, tax, contact를 재배치할지 검토한다.
- [] 블로그 콘텐츠를 TS 문자열에서 MDX 또는 CMS로 옮길지 검토한다.
- [] localStorage 기반 인증을 운영용 인증 체계로 교체할지 결정한다.
- [] contact form에 rate limit, captcha, anti-spam을 추가할지 검토한다.
- [] 클라이언트 컴포넌트 경계를 더 줄여 hydration 범위를 최소화한다.

15. 추가 확인이 필요한 질문

질문은 최소화하고, 확인이 필요한 핵심만 정리한다.

1. 사용자가 원래 의도한 "기존 Pages Router 분석 md" 원문이 별도로 있는가?
2. 현재 Vercel의 Production Branch와 Custom Domain 설정은 어떻게 되어 있는가?
3. /DJ_Play_List 는 이미 외부에 공개된 운영 URL인가?
4. 로그인/회원가입/프로필 기능은 데모용인가, 실제 운영 기능인가?
5. 연락 폼에는 rate limit, captcha, anti-spam이 필요한 운영 트래픽이 있는가?

부록: 현재 확인된 실제 근거

실제 저장소에서 확인된 대표 근거는 다음과 같다.

- App Router 루트 layout 존재: `app/layout.tsx`
- App Router 홈 페이지 존재: `app/page.tsx`
- Route Handler 존재: `app/api/send-email/route.ts`
- 동적 App Router 라우트 존재: `app/blog/[slug]/page.tsx`
- `generateStaticParams` , `generateMetadata` 사용 확인
- `next/navigation` 사용 확인
- `next/head` , `next/router` , `getStaticProps` , `getServerSideProps` , `getStaticPaths` 미확인
- `pages/` 디렉터리 미확인

최종 판단

이 저장소는 Pages Router -> App Router 전환 대상이라기보다, 이미 App Router로 운영 중인 상태에서 구조 안정화, SEO 보강, 배포 전략 준비가 필요한 프로젝트다.